第 9 章練習解答　虛擬記憶體

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS09-E01　Reference string 替換

1. FIFO=9、LRU=10、OPT=7、CLOCK=9 faults。前三頁從空 frame載入；其後 eviction/hit順序依 policy metadata，不能共用同一 resident tuple。
2. FIFO victims為`-, -, -, 1, 2, 3, 4, -, -, 1, 2, -`。第8次page1與第9次page2命中，FIFO不因hit調整載入隊列。
3. 同一 string 的 FIFO在3 frames為9 faults，4 frames為10 faults；增加frame反而增加 fault，就是 Belady anomaly。
4. OPT對每個 reference string仍有固定最小 eviction數；異常來自非stack policy的 FIFO。精確LRU每次access需更新排序，counter或stack實作增加成本且需一致性維護，因此實務常用reference-bit近似。

四政策逐次狀態如下。FIFO按載入先後、LRU由最舊到最新，CLOCK則列實體slots；F表示fault、H表示hit。

t/page	FIFO	LRU	OPT	CLOCK
1/1	[1]F	[1]F	[1]F	[1]F
2/2	[1,2]F	[1,2]F	[1,2]F	[1,2]F
3/3	[1,2,3]F	[1,2,3]F	[1,2,3]F	[1,2,3]F
4/4	[2,3,4]F	[2,3,4]F	[1,2,4]F	[4,2,3]F
5/1	[3,4,1]F	[3,4,1]F	[1,2,4]H	[4,1,3]F
6/2	[4,1,2]F	[4,1,2]F	[1,2,4]H	[4,1,2]F
7/5	[1,2,5]F	[1,2,5]F	[1,2,5]F	[5,1,2]F
8/1	[1,2,5]H	[2,5,1]H	[1,2,5]H	[5,1,2]H
9/2	[1,2,5]H	[5,1,2]H	[1,2,5]H	[5,1,2]H
10/3	[2,5,3]F	[1,2,3]F	[2,5,3]F	[5,3,2]F
11/4	[5,3,4]F	[2,3,4]F	[5,3,4]F	[5,3,4]F
12/5	[5,3,4]H	[3,4,5]F	[5,3,4]H	[5,3,4]H

檢查。 逐列記錄 frames 與 victim，避免只報總數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答　虛擬記憶體 (續)

OS09-E02　有效存取時間

1. EAT = (1-p)×80 + p×5,000,000 ns。80 + p(4,999,920) <= 100。
2. p <= 20/4,999,920，約 4.00006×10^-6，約每 250,000 次存取最多 1 次 fault。
3. 80 + p×9,999,920 <= 100，p <= 2.000016×10^-6，約每 500,000 次最多 1 次。

檢查。 毫秒要先換成奈秒，再作比率。

OS09-E03　Working set 與 thrashing

1. 20+24+28=72 > 64，需要量超過可用量約 8 frames。
2. 不一定已 thrashing；working set 是窗口內引用上界，有些頁可能低頻且可共用，實際 fault rate 與 locality 仍要量測。
3. 可降低多程式度、調整替換與配置、增加 memory、設定 admission control，或改變工作集/回收政策。

檢查。 模型預測壓力，runtime fault/latency 證實是否真的 thrashing。

OS09-E04　Demand fault 與 COW fault 完整路徑

1. 先驗證VMA/read權限，撤銷P1 PTE並完成舊P1 TLB失效，再等dirty writeback完成、read P5、公布新mapping與架構要求的維護、retry。1次fault、1次disk write、1次disk read；I/O不重疊且忽略其他開銷為16ms。本題把撤銷舊translation計為一個維護階段；實際TLB指令/遠端shootdown數依架構與CPU快取狀況另計。
2. clean P2不需卸載前writeback，polling/command overhead忽略時只有8 ms read；fault count仍是1，disk writes=0、disk reads=1，並且不可把節省的write誤算成zero-copy。
3. 原translation存在，write permission因COW被暫時關閉。handler配置F23，先複製4096bytes而尚未公布新mapping；再令child PTE指F23可寫、維護TLB後重試store42。F17映射數2→1、值41不變，F23映射數1、值42 dirty。parent PTE可繼續唯讀，將來parent write fault確認私有獨佔後才可免複製提升。
4. handler讀入P5卻未安裝PTE/TLB，原instruction會再次fault；或釋放仍由victim mapping使用的frame，會讓其他process重用並造成資料破壞。另一錯誤是未等dirty writeback完成就覆蓋victim。

檢查。 Demand與COW fault的判定原因、資料來源、frame變化、權限與retry必須逐項一致。

第 9 章練習解答　虛擬記憶體 （續）

OS09-E05　Frame allocation 與 local/global victim

1. raw=10.667/21.333/32.000，floor=10/21/32合計63；剩1frame最大fraction是A 0.667，所以最終A/B/C=11/21/32。
2. local只看A，移除A2並載入A7，A={1,7,3}、B={4,5}、C={6}。global移除B4，將它的frame交給A7，A={1,2,3,7}、B={5}、C={6}。兩者都維持總數6，只有global改變每個process的resident數。
3. global移除了B4，因此B下一次存取4便fault。A3仍resident，不能為了讓A保持3頁而再刪一頁；也沒有任何請求可讓B憑空載入8。
4. 公平性不能製造frame。架構若要求某指令重啟期間有若干code/operand頁同時駐留，就須滿足該下界或改變執行/重啟機制；不足可暫停或拒絕admission。working-set大小是效能需求，不等於所有process固定的correctness下界；fault handler核心頁由系統另行保證。

檢查。 Quota、victim pool、resident state與下一個fault必須連成同一因果鏈。

OS09-E06　Working set 與 PFF 控制

1. A={1,2,4,5}、B={6,7,8}、C={9}；WSS=4/3/1，D=8。m=6，若這些頁都維持active demand，working-set模型預測配置不足並有thrashing風險。
2. FFFFF→1.0>0.4，請求/分配frame；FHHHH→0.2介於門檻，保持；HHHHH→0<0.1，可在memory pressure下考慮回收。雙門檻形成hysteresis，避免在單一線附近反覆分配/回收。
3. 降低多程式度、換出process、重新分配frames，或拒絕新的admission；不能只提高排程priority，frames不足時short-term CPU scheduling不會製造resident pages。
4. A/B/C的最近4次窗口集合可大於6，但部分頁已不再被使用、被共享、或在sleep/blocked；若實際fault rate低、CPU utilization高、I/O queue平穩，就沒有thrashing證據。相反，高fault rate+低CPU utilization+paging queue成長才能支持診斷。

檢查。 Working set是需求預測，PFF與CPU/I/O證據檢驗是否真的thrashing。

第 9 章練習解答　虛擬記憶體 （續）

OS09-E07　容量單調性與離線最優的證明

1. 按最後access從新到舊排所有已見頁；每次把請求頁移到首位。對每一容量k，歸納證明resident恰為前k項：hit只改順序，miss插首並擠掉最舊第k項。因此Rk是R(k+1)的子集；容量k+1若miss，請求頁亦不在子集Rk。每個時刻的fault indicator皆不增，加總得F(k+1)≤Fk。
2. FIFO的順序由各容量實際載入時刻決定；容量改變會改變先前fault，沒有同一個全域stack可取前k。CLOCK的R-bit與hand也依容量及掃描而改變，近似近期性並非精確LRU包含不變量。
3. 在共同resident上首次分歧，最優排程移除p，OPT移除q且next(q)≥next(p)。修改排程先留p捨q；到首次要求p之前不會先要求q，可追隨共同頁的替換。若原排程先移除q就可對齊；否則它到p要fault，修改排程已有p，可用省下的這一次傳輸載入q並對齊，成本不增。把提前讀入延至首次要求不增加成本，故存在此步遵循OPT的合法最優排程。反覆消除首次分歧。若dirty eviction另付write、或頁傳輸不同權重，這個一比一傳輸交換不再保證成本不增。

檢查。 有限reference枚舉可檢查模型，不能取代對任意string的歸納與交換證明。

OS09-E08　COW與舊TLB的隔離反例

1. CPU1可能用舊可寫TLB把store送到F1，破壞剛讀入的P5。必須先撤銷P1 PTE、完成必要本地失效與遠端shootdown確認、阻止新舊存取，等dirty writeback/I/O與其他引用安全結束，才重用F1、安裝P5並重試。
2. 初始兩者指F40/7。P2寫9後只P2改指新F53/9，P1仍F40/7，cache與file仍7，copy4096bytes。P1之後寫8才另配置F52/8，P2仍F53/9，累計copy8192bytes；private修改不寫回原file。
3. 不必。parent PTE可以繼續W=0/COW，之後parent write fault時重新確認是可獨佔的private anonymous頁，沒有file-cache共享、額外pin或其他引用限制，再只調權限而免複製。單憑一個mapping計數不能把所有共享file頁變成可寫。

檢查。 先檢查mapping與translation可達性，再判斷frame能否安全重用。

第 9 章練習解答　虛擬記憶體 （續）

OS09-E09　Linux 3.2 reclaim的條件辨析

1. P1 activate，P2在mapping/refcount可移除後reclaim，P3成功啟動非同步writeback，P4 cull至unevictable，P5 defer；只有P2 free+1。每個候選必須真的走到最後移除所有權才能增加free。
2. v3.2對dirty file page限制由kswapd且priority<DEF_PRIORITY−2才在本分支pageout；DEF_PRIORITY12故priority10被擋。direct reclaim不是kswapd也被擋，保留並安排其他writeback，不得仍把P3畫成已啟動此pageout。
3. page_check_references設PageReferenced並回PAGEREF_KEEP；首次file reference不保證activate。若已有PageReferenced或是anon的PTE reference才有不同分支。
4. 不一樣。mlock形成unevictable條件；一般額外pin可使最後reference/mapping移除失敗，頁仍可能在一般LRU保留。writeback完成後須重查dirty、writeback、新引用及pin；若被重新寫dirty或仍有reference，仍不能重用。

檢查。 結果必須連到page type、reference來源、scan context與最終所有權，而非只看一個dirty bit。

OS09-E10　詳解：分開追蹤引用與不同頁

(1) t=4..8 的視窗依序為 [1,2,1,3]、[2,1,3,1]、[1,3,1,2]、[3,1,2,4]、[1,2,4,1]。WS 依序為 {1,2,3}、{1,2,3}、{1,2,3}、{1,2,3,4}、{1,2,4}；WSS 為 3,3,3,4,3。t=5 移出一次頁 1 的引用，又加入一次頁 1；其計數仍為 2，不得移除。

(2) 存取前 r 為 首次、首次、2、首次、2、3、首次、3。容量 2：F,F,H,F,H,F,F,F，共 6 次；容量 3：F,F,H,F,H,H,F,H，共 4 次。第 6 與 8 次都恰好位於 r=3，只有容量 3 能命中。

(3) t=7 後 WS={1,2,3,4}，LRU 3 的順序為 [4,2,1]，頁 3 已淘汰。兩次使用頁 p 間的每個不同頁都排在 p 前方，重複使用同一個其他頁不會再多排一頁，因此 r=d+1。駐留的前 k 頁包含 p 當且僅當 r≤k，即 d<k。首次引用另判為缺頁。這個模型下增加 k 不會把既有命中變成缺頁，但增加視窗長度只是改變工作集的需求估計，並未直接配置 frame。

第 9 章練習解答　虛擬記憶體 （續）

原創驗算程式附錄: page_replacement.py

```
"""FIFO, LRU, and offline OPT page-replacement simulation.

Run:
    python page_replacement.py
"""


def run_policy(
    reference: list[int], frames: int, policy: str
) -> tuple[int, list[tuple[int, tuple[int, ...], bool, int | None]]]:
    resident: list[int] = []
    faults = 0
    trace: list[tuple[int, tuple[int, ...], bool, int | None]] = []
    for index, page in enumerate(reference):
        victim = None
        if page in resident:
            if policy == "LRU":
                resident.remove(page)
                resident.append(page)
            trace.append((page, tuple(resident), False, None))
            continue
        faults += 1
        if len(resident) < frames:
            resident.append(page)
            trace.append((page, tuple(resident), True, None))
            continue
        if policy == "FIFO":
            victim = resident.pop(0)
        elif policy == "LRU":
            victim = resident.pop(0)
        elif policy == "OPT":
            future = reference[index + 1 :]
            victim = max(
                resident,
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章練習解答　虛擬記憶體 （續）

```
                key=lambda item: future.index(item) if item in future else len(future) + 1,
            )
            resident.remove(victim)
        else:
            raise ValueError(f"unknown policy: {policy}")
        resident.append(page)
        trace.append((page, tuple(resident), True, victim))
    return faults, trace


def simulate(reference: list[int], frames: int, policy: str) -> int:
    faults, _ = run_policy(reference, frames, policy)
    return faults


def main() -> None:
    reference = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
    result = {policy: simulate(reference, 3, policy) for policy in ("FIFO", "LRU", "OPT")}
    assert result["FIFO"] == 9
    assert result["LRU"] == 10
    assert result["OPT"] == 7
    lru_faults, lru_trace = run_policy(reference, 3, "LRU")
    victims = [entry[3] for entry in lru_trace if entry[3] is not None]
    assert lru_faults == 10
    assert victims == [1, 2, 3, 4, 5, 1, 2]
    assert [entry[3] for entry in lru_trace[9:]] == [5, 1, 2]
    print("faults", result)
    print("lru_trace", lru_trace)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study